Desafio Técnico – Eng. Front-End (Pleno)

🎯 Objetivo

Avaliar a capacidade de desenvolver uma aplicação moderna em React com foco em:

- Reutilização de componentes
- Otimização de chamadas e efeitos
- Boas práticas de UX
- **Manipulação e persistência de dados em estado global**

🛠️ Desafio: "Triskin Store Admin"

Funcionalidades:

1. Listagem de Produtos

- Requisição para API mock (pode usar MockAPI, JSON Server, etc.)
- Cada item exibe:
  - Nome
  - Preço
  - Status (Ativo/Inativo)
  - Botão **"Adicionar ao Carrinho"**

2. Carrinho de Compras (Estado Global)

- Quando o usuário clica em **“Adicionar ao Carrinho”**:
  - O produto é salvo em estado global (Zustand, Context API, Jotai ou Redux)
  - A quantidade do item é incrementada se já existir

- O **ícone do carrinho na header** mostra:
    - Quantidade total de itens
    - Badge com número atualizado em tempo real
- Criar uma **página de "Carrinho"** com:
    - Lista de produtos adicionados
    - Possibilidade de remover/alterar quantidade
    - Totalizador de valor

3. Formulário de Edição (da listagem)

- Editar dados do produto (modal ou rota separada)
- Salvar altera dados locais e sincroniza com back-end fake
- Desabilitar botão e mostrar feedback durante o envio

4. Busca com debounce

- Campo de busca (500ms debounce)
- Filtra lista por nome do produto

5. UX Responsiva e Feedback

- Loaders e feedbacks para:
    - Estado de carregamento global e por componente
    - Submissão de formulário
    - Adição ao carrinho

6. Componentização obrigatória

Separar os seguintes componentes:

- ProductList, ProductItem
- EditProductModal ou EditPage
- SearchInput
- CartIcon, CartPage
- Loader, ErrorMessage

Critérios Avaliados

Componentização

- Componentes limpos, reutilizáveis e coesos
- Separação por responsabilidade

Otimização de Dados

- Redução de useEffect
- Memoização onde necessário
- Controle inteligente de re-render e chamadas

Estado Global

- Gerenciamento global eficiente do carrinho
- Consistência do estado entre componentes e páginas
- Simplicidade e escalabilidade

UX

- Interações bloqueadas durante envio/carregamento
- Feedback visual claro para o usuário
- Prevenção de múltiplos cliques

Entrega Esperada

- Repositório público no GitHub
- README com:
    - Instruções de execução
    - Tecnologias e bibliotecas usadas
    - Explicações das estratégias de otimização e estado global

💼 Stack Obrigatória

- React + Hooks
- TypeScript
- Styled Components, Tailwind ou CSS Modules
- Axios ou Fetch
- Gerenciador de estado (Context, Zustand, Redux, Jotai)

💎 Extras Valorizados (Bônus)

- React Query ou SWR
- Testes com Vitest/Jest + RTL
- Design System leve (ex: com Radix ou Shadcn)
- Deploy em Vercel ou Netlify